

1. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

CODE:

```
nm FICD 3.09 File: mm.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>
#include <errno.h>

int main() {
    char path[PATH_MAX];
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX];

    // Get current working directory
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        return 1;
    }

    strcpy(previous_dir, current_dir);

    printf("Current Directory: %s\n", current_dir);

    // Ask user for directory
    printf("Enter directory path to change to: ");
    if (fgets(path, sizeof(path), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    // Remove newline
    path[strchr(path, '\n')] = '\0';

    // Validate path
    if (access(path, F_OK) != 0) {
        printf("Error: Path does not exist.\n");
        perror("access");
        return 1;
    }

    // Check whether it is a directory
    if (access(path, X_OK) != 0) {
        printf("Error: Cannot access this directory.\n");
        perror("access");
        return 1;
    }

    // Change directory
    if (chdir(path) != 0) {
        printf("Error: Unable to change directory.\n");
        perror("chdir");
        return 1;
    }

    // Display new working directory
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        return 1;
    }

    printf("\nDirectory changed successfully!\n");
    printf("Previous Directory: %s\n", previous_dir);
    printf("New Working Directory: %s\n", current_dir);

    // Ask whether to return to previous directory
    printf("\nDo you want to return to the previous directory? (y/n): ");

    char choice;
    scanf("%c", &choice);

    // Get Help
    // Exit
    // WriteOut
    // Justify
    // Read File
    // Where is
    // Prev Pg
    // Next Pg
    // Out Text
    // UnOut Text
    // Cur Pos
    // To Spell
```

OUTPUT:

```
Last login: Thu Sep 17 16:47:08 on tty000
greenikhilreddy@Strees-MacBook-Pro ~ % nano mm.c
greenikhilreddy@Strees-MacBook-Pro ~ % gcc mm.c -o mm
greenikhilreddy@Strees-MacBook-Pro ~ % ./mm
Current Directory: /Users/greenikhilreddy
Enter directory path to change to: exist
Error: Path does not exist.
access: No such file or directory
greenikhilreddy@Strees-MacBook-Pro ~ %
```

2. To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

CODE:

```
10W FICD 5.09 File: 161.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

// Function prototypes
void shell_cd(char **args);
void shell_pwd(char **args);
void shell_exit(char **args);
void shell_help(char **args);

// Structure for dispatch table
typedef struct {
    char *command;
    void (*function)(char **args);
} builtin;

// Dispatch table
builtin builtins[] = {
    {"cd", shell_cd},
    {"pwd", shell_pwd},
    {"exit", shell_exit},
    {"help", shell_help}
};

#define NUM_BUILTINS (sizeof(builtins) / sizeof(builtins[0]))

// Find built-in command
int find_builtin(char *command) {
    for (int i = 0; i < NUM_BUILTINS; i++) {
        if (strcmp(command, builtins[i].command) == 0) {
            return i;
        }
    }
    return -1;
}

// cd command
void shell_cd(char **args) {
    if (args[1] == NULL) {
        fprintf(stderr, "cd: missing directory\n");
        return;
    }
    if (chdir(args[1]) != 0) {
        perror("cd");
    }
}

// pwd command
void shell_pwd(char **args) {
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd");
    }
}

// exit command
void shell_exit(char **args) {
    printf("Exiting shell...\n");
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where Is Next Pg UnCut Text To Spell
```

OUTPUT:

```
green@hillelreddy@Stress-MacBook-Pro ~ % nano 161.c
green@hillelreddy@Stress-MacBook-Pro ~ % gcc 161.c -o 161
green@hillelreddy@Stress-MacBook-Pro ~ % ./161
=====
MINI DISPATCH SHELL
=====
Type 'help' for available commands.
myshell$
```

3. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

CODE:

```

1  UN FICD 5.09
2  File: hh.c
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7  #include <unistd.h>
8  #include <limits.h>
9
10 #define MAX_INPUT 100
11
12 void display_current_directory()
13 {
14     char cwd[PATH_MAX];
15     if (getcwd(cwd, sizeof(cwd)) == NULL)
16     {
17         perror("getcwd");
18         return;
19     }
20     printf("Current Directory: %s\n", cwd);
21 }
22
23 void save_state()
24 {
25     char cwd[PATH_MAX];
26     FILE *fp;
27     if (getcwd(cwd, sizeof(cwd)) == NULL)
28     {
29         perror("getcwd");
30         return;
31     }
32     fp = fopen("shell_state.txt", "w");
33     if (fp == NULL)
34     {
35         perror("fopen");
36         return;
37     }
38     fprintf(fp, "Last Working Directory: %s\n", cwd);
39     fclose(fp);
40     printf("State saved successfully.\n");
41 }
42
43 void cleanup()
44 {
45     printf("Cleaning up resources...\n");
46     save_state();
47     printf("Cleanup completed.\n");
48 }
49
50 void exit_shell()
51 {
52     printf("\nExit command received.\n");
53     cleanup();
54     printf("Shell exiting...\n");
55     exit(EXIT_SUCCESS);
56 }
57
58 int main()
59 {
60     Get Help
61     Exit
62     Write Out
63     Justify
64     Read File
65     Where Is
66     Prev Pg
67     Next Pg
68     Cut Text
69     UnCut Text
70     Cur Pos
71     To Spell

```

OUTPUT:

```

1  screen1@hillelreddy@Strees-MacBook-Pro: ~ % nano hh.c
2  screen1@hillelreddy@Strees-MacBook-Pro: ~ % gcc hh.c -o hh
3  hh.c:7:9: warning: "MAX_INPUT" macro redefined [-Wmacro-redefined]
4      7 | #define MAX_INPUT 100
5        | ^
6  /Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/syslimits.h:98:9: note: previous definition is here
7      98 | #define MAX_INPUT 1024 /* max bytes in terminal input */
8        | ^
9  1 warning generated.
10 screen1@hillelreddy@Strees-MacBook-Pro: ~ % ./hh
11
12 =====
13 SIMPLE C SHELL
14 =====
15
16 Available commands:
17 pwd - Display current directory
18 exit - Exit the shell
19 help - Display commands
20
21 myshell$

```

4. To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

UN FICD 5.09File: jj.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 256

/* Validate environment variable name */
int valid_name(const char *name)
{
    int i;

    if (name == NULL || name[0] == '\0')
        return 0;

    /* First character must be alphabet or '_' */
    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    /* Remaining characters: alphabet, digit, or '_' */
    for (i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

/* Process export command */
void process_export(char *input)
{
    char *argument;
    char *equal;
    char *name;
    char *value;

    /* Skip "export" */
    argument = input + 6;

    /* Skip spaces */
    while (*argument == ' ' || *argument == '\t')
        argument++;

    /* Check if argument exists */
    if (*argument == '\0')
    {
        printf("Usage: export NAME=VALUE\n");
        return;
    }

    /* Find '=' */
    equal = strchr(argument, '=');

    if (equal == NULL)
    {
        printf("Error: Invalid export syntax.\n");
        printf("Usage: export NAME=VALUE\n");
        return;
    }

    /* Separate name and value */
    *equal = '\0';
    name = argument;
    value = equal + 1;
}
```

Get HelpExitWriteOutJustifyRead FileWhere isPrev PgNext PgCut TextUnCut TextCur PosTo Spell

CODE:

```
screen@hilleddy@Stress-MacBook-Pro ~ % nano jj.c
screen@hilleddy@Stress-MacBook-Pro ~ % gcc jj.c -o jj
screen@hilleddy@Stress-MacBook-Pro ~ % ./jj
=====
EXPORT ENVIRONMENT SHELL
=====
Commands:
  export NAME=VALUE
  test NAME
  env NAME
  exit

myahell$ nikhil
Invalid command.
myahell$ exit
Exiting shell...
screen@hilleddy@Stress-MacBook-Pro ~ % nano jj.c
screen@hilleddy@Stress-MacBook-Pro ~ % █
```